ICT-1203: Object Oriented Programming Lab

Project Name: Personal Expense Tracker

Submitted To:

Dr.Shamim Al Mamun

Professor

Institute Of Information technology

Jahangirnagar university

Submitted By:

Nanjiba Nawar Hoque(1831)

Mossammed Sinha(1832)

Refaya Tul Islam(1833)

Farhana Jannat Nadiha(1834)

Noushin Sultana(1835)

Abir Howladar(1836)

Ahsan Kabir(1837)

Md.Ashraful Rahman(1838)

Md.Mahamudul Hassan Antor(1839)

Nahian Hossain Arman(1840)

Submission Date: **19 August, 2026**

Acknowledgement:

First of all, thanks to our course teacher, Dr. Shamim Al Mamun,for guiding us through this project from start to finish. There were a few points where we were genuinely stuck on how to move forward, and the feedback we got in class or during discussion hours usually ended up being the thing that pushed us in the right direction. We appreciate the patience shown toward a group of students still figuring things out.

We'd also like to thank the Institution Of Information Technology at Jahangirnagar University for giving us the chance to work on something like this as part of our coursework. It's one thing to study concepts in a classroom and another to actually sit down and build them into something that works, even if imperfectly, and this project gave us that opportunity.

And of course, credit goes to every member of the group. Between coordinating schedules, dividing the work, and going back and forth on how certain features should behave, this was very much a joint effort. A few late-night sessions and more than one redesign later, here we are.

Abstract:

It is important to manage both one's income and expenses if proper financial control is to be maintained. Yet it can become difficult to keep record of daily transactions by hand, particularly when the number of such transactions rises. In order to solve this problem, the project offers a simple Personal Expense and Income Management System which has been developed using Java.

Users can record their income and expenses and keep all the information in one place. The system offers various choices when adding transactions, when looking at the recorded information, when searching for expenses, and when viewing the records of both income and expenses. A graphical user interface is employed in order that users may interact with the system without having to work directly with the program code.

The project is mainly concerned with applying object-oriented programming concepts in a practical application, using different classes to represent transactions, expenses, income, and the various data management operations. The system as a whole offers a simple method of keeping financial records and shows how Java can be made use of in developing a useful desktop-based application.

Introduction:

It is common for people to face difficulties when managing their money. They usually have a number of sources of income and suffer from many small expenses during the day. If these transactions are merely written down in notebooks, in messages, or in personal notes, then it becomes hard to locate earlier records or to know where the money has been spent.

A computerized expense management system can simplify this process by organizing the income and expenditure information. Rather than keeping their records separately and having to do so by hand, users can input their transactions into one system and then access them when needed.

The Personal Expense and Income Management System was created using Java, and the programme includes a graphical interface enabling users to add income, add expenses, view the existing records,

and search for specific expense information. The system is made up of a number of classes in order that different responsibilities may be managed separately.

The project also provides practical experience in important Java concepts including classes and objects, object-oriented design relating to inheritance, data handling, event-driven programming, and the development of graphical user interfaces. Because of this, the project is not only a useful basic financial record-keeping application but also shows how programming concepts can be combined to solve a real-life problem.

Problem Statement:

Manually keeping record of personal financial transactions is time-consuming and very inconvenient. People might forget about small expenses, lose their written records, or have trouble locating a specific transaction when they need it. The more transactions there are, the harder it becomes to keep separate records of income and expenses.

A frequently encountered issue is that records which are kept manually do not offer a simple method for searching or organising the information. For instance, if someone wishes to locate an earlier expense, they might have to go through a large number of entries. As a result, financial management becomes less efficient.

There therefore arises a requirement for a simple system which can store information about income and expenditure in an organised way and which allows users to easily access the records. The project in question seeks to offer this solution by means of a desktop application based on Java.

Motivating Existing problem:

The major motivation that triggered the development of this project lies in the problems faced by individuals in keeping track of their everyday financial transactions. In ordinary situations, there may be multiple smaller financial transactions like expenditure on transportation, food, buying goods, studying and various other expenses. Unless kept in proper records, it might become hard to find out the total amount spent.

Methods like using notebooks and even text-based notes help in storing information but lack ease of use when the user wants to access some particular transaction or deal with a large amount of data. The possibility of information being lost increases if all is done manually.

This problem motivated us to design a simple computerized program in which all transactions of money could be added and managed from one place. With separate modules for adding and viewing financial details, the system makes the whole process organized.

Another motivating factor was the chance to implement our programming skills learnt in the classroom in solving some practical issue. Rather than applying Java concepts just for programming tasks, we tried to apply various concepts together for a practical application.

Objectives & Scope:

The Personal Expense Tracker project was developed with a clear set of objectives, and it also has a defined scope for future growth.

Recording Transactions:

The main objective is todesign a system that allows a user to easily record andtrack daily income and expenses, so that all financial

activity is available in one place.

Category-wise Tracking:

Expenses are organized intocategories such as Food, Education, Health, Entertainment and Others, so that spending patterns can be analyzed at a glance through the dashboard.

Balance Overview:

The system gives the user a clear, real-time view of the current balance, using the getTotalExpense() and get_total_Income() methods of the Reposatory class.

Persistent Storage:

The objective is to keep records safe between sessions by saving and loading data through a CSV file using the generateCSV() and loadCSV() methods.

Scope:

At present, the system works as a single-user, desktop-based application built with Java and Java Swing. It can be extended in the future by connecting it to a database such as MySQL, which would allow multiple users to maintain their own separate records. The project also has scope to grow into a web or mobile application, and graphical/chart-based visual analysis of spending trends can be added on top of the existing category-wise percentage system.

Project Features:

The application provides several features that together allow the user to manage income and expenses from a single dashboard.

Add Income:

The user can record income by entering the source, amount, date and note through the add_income screen, which creates a new Income object and passes it to the repository.

Add Expense:

The user can record an expense under a chosen category through the addExpense screen, which creates a new Expense object and stores it using addTransaction().

Search:

Transactions can be searched by category for expenses, or by source for income, using the searchExpenseByCategory() and searchIncomeBySource() methods.

Delete:

Incorrect or unwanted entries can be removed from the transaction list using the deleteExpense() and deleteIncome() methods.

The system automatically calculates what percentage of the total expen

Percentage-based Expense Calculation:

se each category represents, using methods such as getFoodPercentage(), getEducationPercentage(), getHealthPercentage(), getEntertainmentPercentage() and getOthersPercentage(). This gives the user a quick visual sense of where most of the money is being spent.

Recent Expenses:

The dashboard shows expenses made within the last 7 days, calculated by the get7daysexpenses() method.

Category-wise Analysis:

The percentage and amount spent in each category (Food, Education, HealthCare, Entertainment, Others) is displayed on the dashboard, based on the percentage methods in the repository.

Budget Overview:

The Total Income, Spent Expense and Current Balance are displayed together in the My Budget panel of the dashboard.

Data Persistence:

All transactions can be saved to and loaded from a CSV file, so that data is not lost when the application is closed.

Technologies & Requirements:

The project was built using standard Java technologies, along with a set of hardware and software requirements needed to run it.

Technologies Used:

Java – the core programming language used to build the whole application.

Java Swing – used to design the Graphical User Interface (GUI), including JFrame, JTextField, JTable and JButton components.

Apache NetBeans IDE – the development environment used to design, build and run the project.

ArrayList – used inside the Reposatory class as the main collection to store every income and expense record.

java.time.LocalDate and DateTimeFormatter – used to handle and validate transaction dates.

File I/O (FileReader, FileWriter) – used to save and load data through the transactions.csv file.

Hardware Requirements:

Processor: Intel Core i3 or higher

RAM: Minimum 4GB (8GB recommended)

Storage: Minimum 500MB of free disk space

Software Requirements:

Operating System: Windows 10 / 11

Java Development Kit (JDK) – latest version

Apache NetBeans IDE 30

System Design & Architecture:

The system is designed following Object-Oriented Programming (OOP) principles, and it is organized into clear layers so that the GUI, data storage and calculation logic remain separated.

Transaction (Base Class):

An abstract class that stores the common attributes amount, date and Note, and defines the abstract methods calculateImpact() and getType(), which every subclass must implement.

Expense and Income (Subclasses):

Both classes extend Transaction. Expense adds a category field and overrides calculateImpact() to return a negative value, while Income adds a source field and overrides calculateImpact() to return a positive value. This is a practical example of inheritance and polymorphism.

Reposatory (Data Layer): Stores every transaction in a single ArrayList and provides all calculation logic,

such as totals, category-wise percentages, filtering, searching and deleting records.

Architecture Diagram (by layer):

User Interface (Java Swing GUI): addExpense.java, add_income.java, dashbord.java

which connects to.

GUI Layer: Classes such as addExpense, add_income and dashbord collect user input through Swing forms and communicate with the Reposatory object to display up-to-date information.

Model Layer (Transaction hierarchy): Transaction (abstract) leading to Expense and Income

which connects to.

Persistence Layer:

transactions.csv (File Storage).

System Workflow & Use Case:

The workflow of the system describes the step-by-step process a user follows while using the application, from opening it to saving data.

System Workflow:

Start: The user opens the application, and the Main class creates a Reposatory object and passes it to the dashbord, which is then made visible.

Choose an Action:

From the dashboard, the user clicks Add Income or Add Expense to open the corresponding input form.

Enter Details:

The user fills in the required fields, such as amount, date, note, and category or source.

Save Transaction: on submission, a new Expense or Income object is created and added to the Reposatory using addTransaction().

View Updated Dashboard: the dashboard refreshes to show the updated balance and category-wise breakdown.

Search or Delete:

The user can search transactions by category or source, or delete an existing entry when needed.

Save Data:

The user can export all records to a CSV file, ensuring that data remains available the next time the application is opened.

Use Case Diagram (Primary Actor: User):

Add Income

Add Expense

View Dashboard / Balance

Search by Category / Source

Delete Transaction

View Recent Expenses (Last 7 Days)

Save / Load Data (CSV)

Class Diagram s Class Relationship:

The UML class diagram below shows the classes used in the Expense Tracker system — Transaction, Income, Expense, TransactionRepository, TransactionRepositoryImpl, and ExpenseTracker — along with how they are connected to each other.

Figure 1: Class diagram of the Expense Tracker system

Relationship Between Classes UML Notation Meaning

Inheritance Income → Transaction Expense → Transaction Solid line, hollow triangle Income and Expense are specialized types of Transaction and reuse its fields/methods.

Interface Realization TransactionRepositoryImpl → TransactionRepository Dashed line, hollow triangle TransactionRepositoryImpl provides the concrete implementation for the repository contract.

Aggregation TransactionRepositoryImpl ◇— Transaction Solid line, hollow diamond The repository holds a list of Transaction objects, but a transaction can exist independently of it.

Dependency ExpenseTracker ---> TransactionRepository Dashed line, open arrowhead ExpenseTracker uses a TransactionRepository through its interface type only.

Explanation of Each Relationship

• Inheritance: Income and Expense extend Transaction, reusing its id, amount, date, description fields and overriding getSummary() with their own behavior.

• Interface Realization: TransactionRepositoryImpl implements the TransactionRepository interface, providing real logic for add(), remove(), and getAll().

• Aggregation: TransactionRepositoryImpl holds a List — it manages the collection but does not exclusively own each transaction's lifecycle.

• Dependency: ExpenseTracker depends on TransactionRepository (the interface, not the concrete class) to record and read transactions.

Class s Module Description:

Each class in the project has a clearly defined responsibility. The table below summarizes the job of every important class and interface.

Class / Interface Type Responsibility (Job)

Transaction abstract class Common base for every record: id, amount, date, description; declares abstract getSummary().

Income class (extends Transaction) Represents money received; adds source; overrides getSummary().

Expense class (extends Transaction) Represents money spent; adds category; overrides getSummary().

TransactionRepository interface Declares add(), remove(), getAll() — the storage contract.

TransactionRepositoryImpl class (implements interface) In-memory List based implementation of the repository.

ExpenseTracker class (controller) User-facing class exposing recordIncome(), recordExpense(), getBalance(); delegates storage to the repository.

OOP Concept — Abstraction s Encapsulation:

Abstraction

Abstraction means exposing only the essential behavior of an object and hiding its internal details. In this project, Transaction is declared as an abstract class with an abstract method getSummary(). It defines *what* every transaction must be able to do, without fixing *how* — each subclass (Income, Expense) provides its own implementation. Similarly, TransactionRepository is an interface that abstracts away how transactions are actually stored.

public abstract class Transaction { private String id; private double amount; // abstraction: subclasses must define their own summary public abstract String getSummary(); }

Encapsulation

Encapsulation means keeping an object's data (fields) private and exposing controlled access through public getter/setter methods. In Transaction, Income, and Expense, all fields are declared private, and access is only provided through public methods such as getAmount() and getDate(). This protects the internal state from being changed directly from outside the class.

public class Income extends Transaction { private String source; // encapsulated field public String getSource() { // controlled read access return source; } public void setSource(String source) { // controlled write access this.source = source; } }

OOP Concept — Inheritance s Polymorphism:

Inheritance

Income extends Transaction and Expense extends Transaction. This means Income and Expense automatically inherit the amount, date, description fields and the getAmount()/getDate() methods from Transaction, and only need to add what is unique to them (source for Income, category for Expense).

public class Income extends Transaction { private String source; @Override public String getSummary() { return "Income (" + source + "): +" + getAmount(); } }

Polymorphism

Polymorphism allows a Transaction reference variable to point to either an Income or an Expense object, and the correct overridden getSummary() runs automatically at run time. This is called runtime (dynamic) polymorphism, and it lets code such as the repository or ExpenseTracker treat all transactions uniformly through their common Transaction type.

Transaction t1 = new Income("Salary", 30000); Transaction t2 = new Expense("Food", 500); List all = List.of(t1, t2); for (Transaction t : all) { // calls Income.getSummary() or Expense.getSummary() // depending on the actual object type System.out.println(t.getSummary()); }

Method Overriding s Other Java Concepts:

@Override and Method Overriding

When Income and Expense redefine getSummary() with the same signature as in Transaction, this is called method overriding, marked with the @Override annotation. It lets each subclass change the behavior it inherits from the parent class.

Constructors

Constructors initialize an object's fields when it is created. Subclasses call the parent constructor using super(...) to initialize the inherited fields before setting their own.

public Transaction(String id, double amount, LocalDate date, String description) { this.id = id; this.amount = amount; this.date = date; this.description = description; }

public Income(String id, double amount, LocalDate date, String source) { super(id, amount, date, "Income"); // calls Transaction's constructor this.source = source; }

instanceof and Casting

instanceof checks the actual runtime type of an object, and casting converts a general Transaction reference back into its specific subclass so subclass-only members can be accessed. This is used, for example, when calculating the balance in ExpenseTracker.

double balance = 0; for (Transaction t : repository.getAll()) { if (t instanceof Income) { Income income = (Income) t; // downcasting balance += income.getAmount(); } else if (t instanceof Expense) { Expense expense = (Expense) t; balance -= expense.getAmount(); } }

Getters and Private Fields

Every field across Transaction, Income, and Expense is kept private, and public getter methods (getAmount(), getDate(), getSource(), getCategory()) are provided so other classes can read the data without being able to modify it directly — reinforcing encapsulation throughout the project.

Implementation Details:

The Personal Expense Tracker application is implemented using java and java swing . The system manages both income and expense transactions through different GUI modules and a central Reposatory class .

Adding Expense :

First the user enters the expense category , note,date andcost store this input **addExpense class** . Then created expense object **Expense ex = new Expense(...);**

finally the expense object is passed to the reposatory using the **addTransaction()** method **.**

Adding income:

First the user enters the Income source, amount , date and note store this input **add_Income class** . Then created income object finally the income object is passed to the reposatory using the **addTransaction()** method **.**

Transaction Management :

The **Transaction** class works as the common abstract class for transactions . It stores common information such as amount,date and note.It also defines the abstract methods **calculateImpact()** and **getType() .**

Expense Calculation :

The repository calculates the total expense and category wise expenses . It also calculates the percentage of each expense category based on the total expense . This information is used by the dashboard to provide a summary of the users spending .

Searching Expense :

The **Search_Result _dash** class displays expenses according to a selected category . It calls the reposatory's **searchExoenseByCategory()** method and displays the returned data in a jTable.

Displaying Income:

The show_all _ income module displays income records in a table . Users can also select an income record and delete it form the transaction list .

DashBoard :

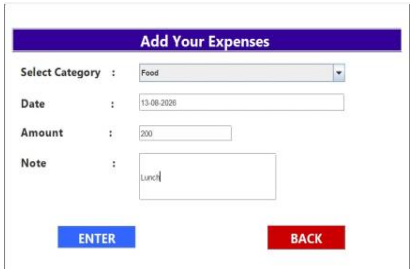
The dashboard class provides the main interface of the application . It connects different modules such as Add Income , Add Expense, Search Expense an d viewing transaction records .

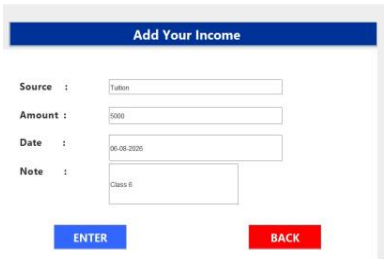
Application Starting Process:

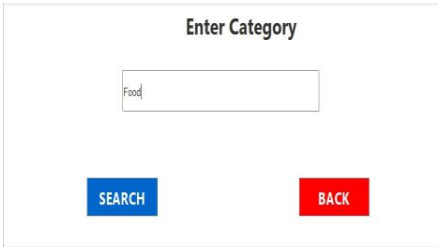
The application starts from the **Main class**. First, a Reposatory object is created. Then this object is passed to the dashbord object, and the dashboard is made visible.

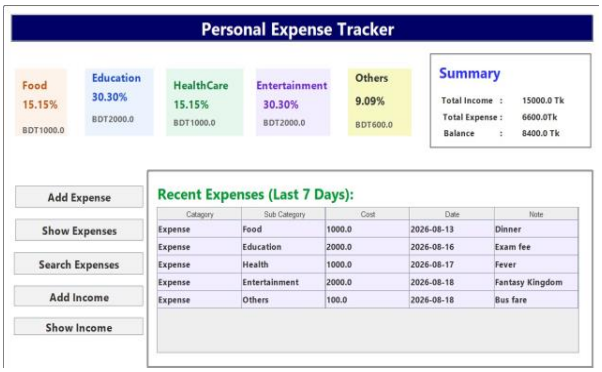
Graphical User Interface(GUI):

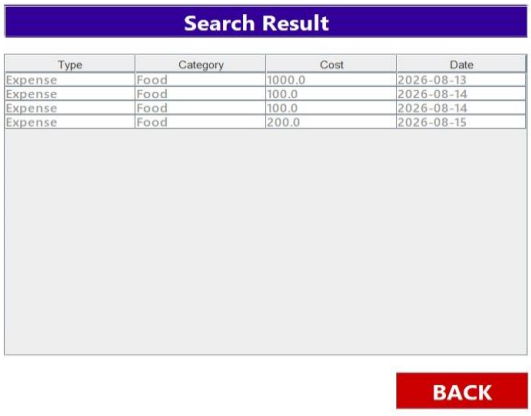
The application contains several graphical screens, including the Dashboard, Add Expense, Add Income, All Income, and Search Result screens. Users can enter the button then show the dashbox .

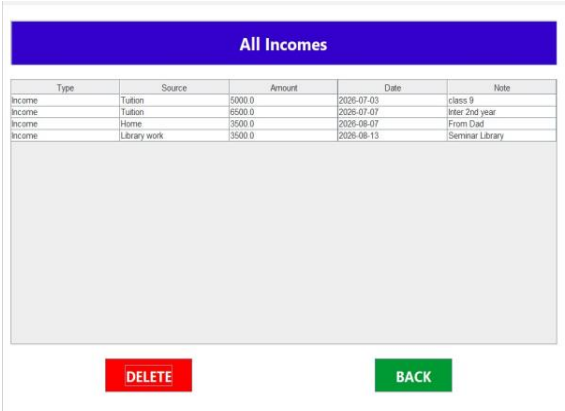












Testing and Results:

Testing is an important part of the Expense Tracker project. The main purpose of testing is to verify whether the implemented features work correctly according to the requirements. Different operations of the application were tested by providing appropriate inputs and checking the resulting out-

Test Case ID	Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Add Income	Enter source, amount, date and note	Income should be added successfully	Income added successfully	Pass
TC-02	Add Expense	Enter category, cost, date and note	Expense should be added successfully	Expense added successfully	Pass
TC-03	View All Income	Open All Income screen	All income records should be displayed	Income records displayed in table	Pass
TC-04	Search Expense	Select an expense category	Matching expenses should be displayed	Matching expenses displayed	Pass
TC-05	Delete Income	Select an income record and click Delete	Selected income should be removed	Selected income removed	Pass
TC-06	Calculate Expense	Add different category-wise expenses	Total and category-wise expenses should be calculated	Expense information calculated	Pass
TC-07	Recent Expenses	Open dashboard/ recent expense section	Recent expenses should be displayed	Recent expenses displayed	Pass

Fig: **Testing table**

The application also provides functionality for displaying recent expenses and calculating expense information through the repository.

Advantages , Limitation Ans Future Improvements :

Advantage:

- Simple and user friendly GUI .
- Easy income and expense management .
- Category based expense tracking .
- Search functionally .
- Recent expense tracking .
- Uses OOP concepts .
- Parcentage based expense system .

Limitation :

- No permanent database storage .
- Limited data validation .
- No advanced reports .

Future Improvements:

- Add a database such as MySQL.
- Add user login and registration.
- Add monthly/yearly reports.
- Add income vs expense charts.
- Add budget limits.
- Add automatic data backup.
- Add stronger input validation.
- Add export to PDF/Excel.

Conclusion:

The Expense Tracker is a Java-based desktop application designed to simplify personal income and expense management.
The project demonstrates important OOP concepts such as Abstraction, Inheritance, Encapsulation, Polymorphism and Method Overriding through the Transaction and Income structure. It also provides practical features such as adding transactions, searching expenses, viewing income and calculating category-wise expenses.
Overall, the project provides a practical application of Java OOP concepts in a real-world problem.